CSCI455 Assignment 3: Pre-training, Fine-tuning, & RAG for Bug Fixing

Jack Stawasz

April 29th, 2026

Repo Link: https://github.com/JackStawasz/WM-Code/tree/main/CSCI-455-GenAI/pretrain-finetune-RAG

Abstract

Assignment 3 investigates whether pre-training a small T5 model improves Java bug fixing, and how a fine-tuned task-specific model compares against retrieval-augmented prompting (RAG) of a larger LLM. Three pipelines are evaluated: Pipeline A (span-corruption pre-training on 50K CodeSearchNet methods, then fine-tuned), Pipeline B (same T5 architecture fine-tuned from random init), and a RAG pipeline that prompts `Qwen2.5-Coder-1.5B-Instruct` with CodeBERT-retrieved exemplars. Counterintuitively, Pipeline B (CodeBLEU 86.82) outperforms Pipeline A (64.51), indicating the small corpus is insufficient for useful transfer. RAG presents a clear trade-off: zero training cost for orders-of-magnitude greater per-sample latency.

Introduction & Training Details

This assignment compares two paradigms for Java bug-fixing: (i) training a small task-specific T5 model, and (ii) prompting a frozen larger LLM with retrieved context. The first paradigm splits into two sub-pipelines that isolate the effect of pre-training: Pipeline A loads a span-corruption pre-trained checkpoint before fine-tuning, while Pipeline B fine-tunes an identical T5 architecture from scratch. The second paradigm uses `Qwen2.5-Coder-1.5B-Instruct` as a frozen generator with both 3-shot RAG and a zero-shot baseline. All experiments were conducted on a T4 GPU in Google Colab. Pre-training and each fine-tuning run took ∼1.5–2 hours, while RAG inference dominates wall-clock time at ∼6 seconds per test sample due to Qwen's larger parameter count.

Tokenizer & Model Configuration

A single SentencePiece tokenizer is trained on the 50K-method pre-training corpus using the Unigram subword algorithm with a base vocabulary of 16,284 tokens. The HuggingFace `T5Tokenizer` wrapper appends 100 sentinel tokens (`<extra_id_0>`–`<extra_id_99>`) along with `<pad>`, `</s>`, and `<unk>`, yielding a final vocabulary of 16,384. This same tokenizer is reused by every downstream pipeline—no retraining or embedding re-initialization between stages—so any difference in fine-tuning results is attributable solely to pre-training. Sanity checks confirmed correct round-trip encoding on Java snippets and proper sentinel ID assignment. The T5 model is constructed manually via `T5Config` with `d_model=512`, `d_ff=2048`, `d_kv=64`, `num_heads=8`, and 6 encoder/decoder layers, producing a ∼52.4M-parameter model after `resize_token_embeddings(len(tokenizer))`. Special-token IDs (`decoder_start`, `eos`, `bos`) match the tokenizer's mapping to ensure span-corruption targets and seq2seq decoding behave correctly.

Pre-training & Fine-tuning Pipelines

Pre-training (Pipeline A): The corpus is sampled from the Java split of CodeSearchNet by extracting `whole_func_string` from 50,000 randomly-selected methods (seed 42), filtered to 10–512 tokens. Each example is corrupted at 15% rate by masking spans of consecutive tokens with unique sentinel tokens; the decoder reconstructs the masked spans separated by their sentinels, exactly matching the original T5 objective. Training runs for 3 epochs with batch size 16, AdamW at LR 1×10^{-4}, mixed-precision via `GradScaler`, and gradient clipping at 1.0. Per the assignment specification, no validation split or checkpoint selection is used. Training loss decreases monotonically across all 3 epochs (Fig. 1, left).

Fine-tuning (both pipelines): Both use the CodeXGLUE `code_x_glue_cc_code_refinement` medium split (∼52K train / 6,545 val / 6,545 test `(buggy, fixed)` pairs), truncated to `MAX_SRC_LEN = MAX_TGT_LEN = 256`. Training uses AdamW at LR 1×10^{-4}, batch size 16, 10 epochs, with per-epoch checkpointing for mid-run resumption. Inference uses beam search (`num_beams=4`) on the full 6,545 test instances. Pipeline A loads the pre-trained checkpoint; Pipeline B reconstructs an identical `T5Config` and trains from random initialization. Every fine-tuning hyperparameter is held constant across pipelines, so pre-training is the sole independent variable. Pipeline B's loss falls below Pipeline A's by mid-training (Fig. 1, right), foreshadowing the test-set results.

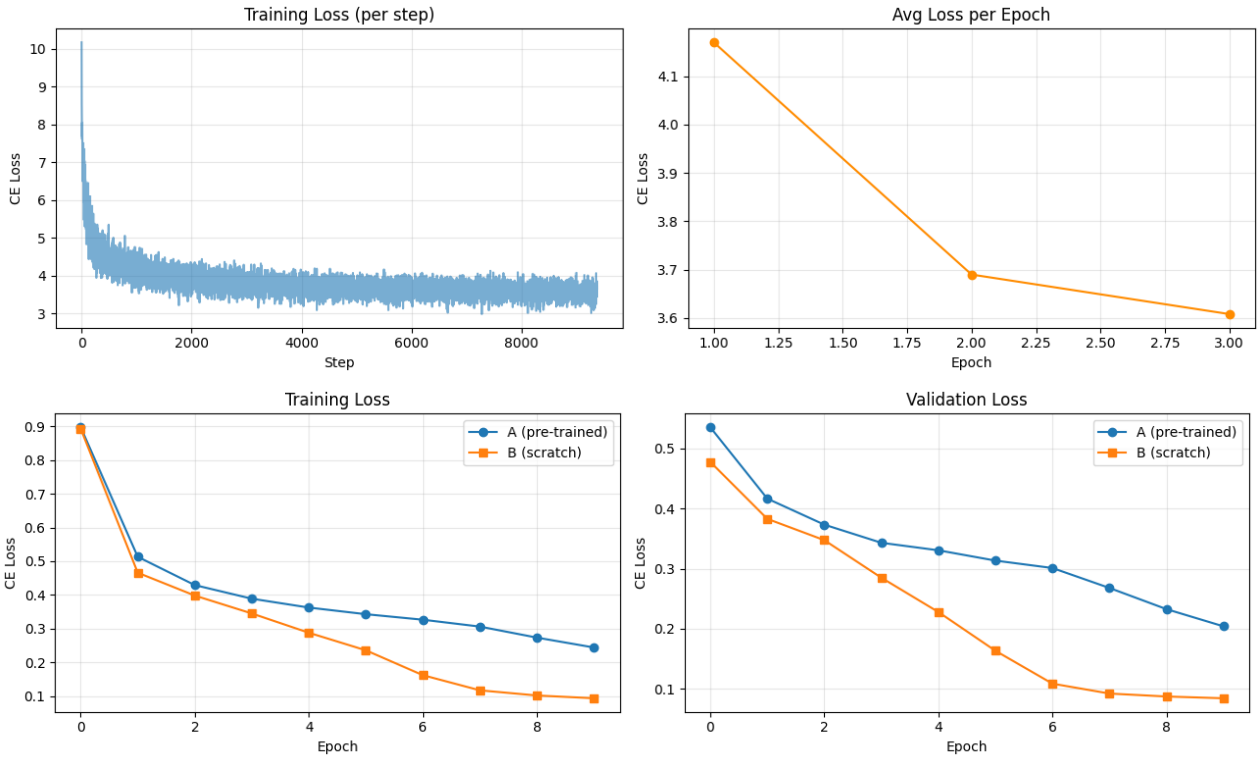

Figure 1: Top: span-corruption pre-training loss (Pipeline A) over 3 epochs on 50K CodeSearchNet methods, from Notebook 1. Bottom: fine-tuning training loss for Pipelines A and B on CodeXGLUE bug fixing, from Notebook 2. Both pipelines share identical fine-tuning hyperparameters; only the initial weights differ.

RAG Pipeline

The RAG retriever uses `microsoft/codebert-base` with mean-pooling over the final hidden states. CodeBERT is preferred over Code2Vec because its transformer architecture produces dense contextual embeddings that capture token-level semantics directly from raw code, well-matched to a setting where small lexical perturbations distinguish bugs from fixes. The training-set buggy methods are encoded once and indexed in a FAISS `IndexFlatL2`; at query time, each test buggy method is encoded and its top-$k = 3$ nearest neighbors are retrieved as few-shot exemplars. The generator is `Qwen/Qwen2.5-Coder-1.5B-Instruct` at `temperature=0` (greedy) for reproducibility. Two prompt formats are evaluated: a 3-shot RAG prompt presenting retrieved `(buggy, fixed)` pairs followed by the query, and a zero-shot baseline supplying only the query with the same instruction header. Outputs are post-processed to strip Markdown fences. To stay within Colab's GPU budget, RAG and zero-shot evaluation use a 500-sample test subset; the fine-tuned models retain full-test-set evaluation.

Results Analysis

Table 1 reports CodeBLEU and exact-match across all four configurations. Pipeline B substantially outperforms Pipeline A on every sub-score, contradicting the typical assumption that pre-training transfers useful representations. Three plausible causes: (i) the 50K-method corpus is too small for span corruption to learn broadly transferable structure; (ii) the span-corruption objective is misaligned with seq2seq buggy→fixed translation, biasing the optimizer toward suboptimal regions; (iii) using identical learning rates for both pipelines may over-perturb Pipeline A's pre-trained weights without warmup. The CodeBLEU breakdown tracks proportionally between pipelines, so the gap is uniform rather than localized. Exact-match remains very low for both—small whitespace or identifier-naming variations break exact equality.

The RAG pipeline occupies a different design point. Despite using a model 25× larger than T5-small, RAG 3-shot achieves CodeBLEU 57.39—below both fine-tuned pipelines—while zero-shot Qwen achieves only 37.83. The 19.6-point lift from zero-shot to 3-shot confirms retrieval contributes genuine signal. A striking pattern is the gap between RAG's n-gram precision (34.00) and structural scores (syntax 73.45, dataflow 82.85): Qwen produces syntactically valid, semantically coherent Java but with different identifiers and phrasing than the reference, deflating n-gram overlap while structural quality remains high.

Configuration	**CodeBLEU**	**n-gram**	**wt. n-gram**	**syntax**	**dataflow**	**Exact**
Pipeline A (pre-train + FT)	64.51	67.73	67.99	61.00	61.30	0.08
Pipeline B (FT only)	86.82	90.50	90.51	82.46	83.82	2.32
RAG 3-shot (Qwen 1.5B)	57.39	34.00	39.25	73.45	82.85	1.40
Zero-shot (Qwen 1.5B)	37.83	3.39	6.01	63.88	78.04	0.00

Table 1: CodeBLEU (with sub-metric breakdown) and exact-match accuracy on the CodeXGLUE bug-fix test set. Fine-tuned pipelines evaluate on the full 6,545 instances; RAG and zero-shot evaluate on a 500-sample subset.

Trade-off: Training Time vs. Inference Time & Error Analysis

The key difference between fine-tuning and RAG is not raw accuracy but *when* compute is spent. Fine-tuning is expensive upfront and cheap at inference: each pipeline took ∼1.5–2 hours on a T4 for 10 epochs over 52K pairs, after which generation runs in <1 s/sample. RAG inverts this: zero training cost, but every call invokes a 1.5B LLM plus a CodeBERT encode and FAISS lookup at ∼6 s/sample. Index-building is fast (<5 min) and label-free. Fine-tuned T5 wins for stable, high-volume deployments—it amortizes training across cheap predictions on commodity hardware—while RAG wins when the task changes often, since rebuilding the FAISS index takes minutes versus hours of retraining on labeled data that may not exist. Pipeline A's underperformance is exacerbated by distribution shift: CodeSearchNet methods differ structurally from CodeXGLUE bug-fix deltas (often single-line), so the inductive bias is partially counterproductive. RAG's dominant failure mode is retrieval drift: when no top-3 exemplar shares the query's bug pattern, Qwen falls back to near-zero-shot generation.

Reproducibility & System Specifications

All experiments ran on Google Colab with a T4 GPU, split across three notebooks: `1_tokenizer_and_pretrain` (tokenizer + Pipeline A pre-training), `2_finetune_and_evaluate.ipynb` (both fine-tuning pipelines + full-test-set CodeBLEU), and `3_rag.ipynb` (FAISS index + RAG/zero-shot inference + metrics). All artifacts are written to `/content/drive/MyDrive/W&M/GenAI/Assignment_3/`. Reproduction requires installing dependencies (README), mounting Drive, and running each notebook top-to-bottom. Key parameters appear in Table 2.

Parameter	Value	Description
`SP_VOCAB_SIZE`	16,384	SentencePiece + 100 sentinels
`PT_EPOCHS` / `FT_EPOCHS`	3 / 10	Pre-train / fine-tune epochs
`LR` / `BATCH_SIZE`	1×10^{-4} / 16	AdamW LR / batch size
Corruption rate	15%	Span-corruption mask rate
`MAX_SRC/TGT_LEN`	256	Token truncation length
`NUM_BEAMS`	4	Beam-search width
RAG retriever / `TOP_K`	CodeBERT / 3	Mean-pool + FAISS, 3-shot
RAG generator / `TEMP`	Qwen-1.5B / 0	Frozen LLM, greedy
`TEST_SUBSET` / `SEED`	500 / 42	RAG eval size / random seed

Table 2: Fixed experimental parameters across all three pipelines.

Discussion & Conclusion

Three lessons emerge. First, pre-training is not a free win: with only 50K methods and 3 epochs of span corruption, Pipeline A's representations did not beat Pipeline B's random initialization—the bet failed at this corpus scale. Second, the fine-tuning vs. RAG comparison is a temporal trade: a 60M T5 at <1 s/sample suits embedded tooling (e.g., IDE linters), while RAG's KB-swap flexibility fits one-off audits or evolving codebases. Third, retrieval quality bounds RAG's ceiling—the zero-shot→3-shot lift directly measures retriever quality, so accuracy depends on the encoder as much as the generator. Future work: (i) larger pre-training corpora to test whether Pipeline A's gap inverts at scale, (ii) warmup or lower fine-tuning LRs tailored to pre-trained initializations, and (iii) hybrid lexical-semantic retrieval to reduce drift.